University Of Engineering and Technology, Lahore
Computer Engineering Department

Lab Evaluation:

Course Name: Object Oriented Programming	**Course Code:** CMPE 221L
Assignment Type: Lab	**Dated:**
Semester: 2nd	**Session:** CYS 2026 Spring
Lab/Project/Assignment #: 6	**CLOs to be covered:** CLO2
Lab Title: Composition & Aggregation	**Teacher Name:** Hafiz Muhammad Mubashar Imran

CLO 2	Apply composition, inheritance and polymorphism and language extension concepts to build classes					
Levels (Marks)	**Level 1**	**Level 2**	**Level 3**	**Level 4**	**Level 5**	**Level 6**
(10)						
					Total	**/10**

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	**9-10**	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	**7-8**	L2	Submitted the lab tasks but have good understanding
Good	**5-6**	L3	Submitted the lab tasks but have weak understanding.
Basic	**3-4**	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	**1-2**	L5	Submitted only one lab task.
Not Acceptable	**0**	L6	Did not attempt

LAB No. 6

Lab Goals/Objectives:

- Basic OO programming in JAVA
- Association in JAVA
- Composition in JAVA
- Aggregation in JAVA

\\\

1. This assignment is to write a DigitalTimer class that creates objects that are digital timers with two fields: hours and minutes. This timer represents 24-hour time, so the hours should take on values 00 through 23 and the minutes should take on values 00 through 59. Your DigitalTimer class should make public the following two constructors and three methods. Test for pre-conditions. The private data in your DigitalTimer class should consist of two Counter objects, one to represent the hours and the other to represent the minutes. Most of the methods from the Counter class should be used to implement the methods for the DigitalTimer class.
Because methods of the Counter class do a lot of the work for you, none of the DigitalTimer methods should be longer than a few lines of code. Test your code with the provided TimerDriver.java

- public DigitalTimer() : to initialise timer to 00:00
- public DigitalTimer(int hours, int minutes) : to initialize timer to hours:minutes check the
- precondition 0 <= hours <=23 and 0 <= minutes <= 59
- public String toString(): to correctly print the digital timer current status
- public void tick(): Increases the time by a minute, wrapping if necessary
- public void set(int hours, int minutes): Sets the current time to hours:minutes. Check the preconditions

Code:

```
class Counter {
   private int value;
   private int max;
   public Counter(int max) {
      this.max = max;
      value = 0;
   }
   public int getValue() {
      return value;
   }
   public void setValue(int newValue) {
      value = newValue;
   }
   public void increment() {
      value = value + 1;
      if (value > max) {
         value = 0;
      }
   }
}
class DigitalTimer {
   private Counter hours = new Counter(23);
   private Counter minutes = new Counter(59);
   public DigitalTimer() {
   }
   public DigitalTimer(int h, int m) {
      set(h, m);
   }
   public void set(int h, int m) {
      if (h >= 0 && h <= 23 && m >= 0 && m <= 59) {
         hours.setValue(h);
         minutes.setValue(m);
      } else {
         System.out.println("Invalid time " + h + ":" + m);
      }
   }
   public void tick() {
      minutes.increment();
      if (minutes.getValue() == 0) {
```

```
            hours.increment();
        }
    }
    public String toString() {
        return String.format("%02d:%02d", hours.getValue(), minutes.getValue());
    }
}
public class Task_01 {
    public static void main(String[] args) {
        DigitalTimer t1 = new DigitalTimer();
        System.out.println("Default: " + t1);
        DigitalTimer t2 = new DigitalTimer(9, 30);
        System.out.println("Start: " + t2);
        t2.tick();
        System.out.println("After tick: " + t2);
        t2.set(23, 59);
        t2.tick();
        System.out.println("23:59 plus one minute: " + t2);
        t2.set(24, 10);
        t2.set(5, 60);
        System.out.println("Still: " + t2);
    }
}
```

Output:

```
Default: 00:00
Start: 09:30
After tick: 09:31
23:59 plus one minute: 00:00
Invalid time 24:10
Invalid time 5:60
Still: 00:00

Process finished with exit code 0
```

Something is Brewing:

2.As discussed, it's time to think about of our walkers as entities, having some attributes and some functionalities. So now reimage your earlier code implementation in object-oriented way and use association. Considering Ali and Hassan to be objects of Directed and Random Walker class correspondingly.

Old problem statement: So lately Ali has been walking as per your instructions. Avoiding obstacles in itspath. Now Hasan wants to start walking with Ali. But Hasan will walk randomly and will avoid the sameobstacles as Ali. Write a code that will simulate both the walk patterns of Ali and Hasan simultaneously.

Output can be of the form

Starting: Ali's Location is = 0 : 0 Hassan's Location is = 0 : 0

Hasan has Random Movement, Ali follows order

ALi: Where should i go now: v

Ali is at = 0 : -1

Hasan is at = 0 : 1

ALi: Where should i go now: ^

Ali is at = 0 : 0

Hasan is at = 0 : 2
ALi: Where should i go now: ^
Ali is at = 0 : 1
Hasan encounters hurdle at [1:2]

Code:

```
import java.util.Random;
import java.util.Scanner;
class Field {
    private int[][] hurdles = { {1, 2}, {-1, 1}, {2, -1}, {0, -3}, {-2, -2} };
    public boolean isHurdle(int x, int y) {
        for (int i = 0; i < hurdles.length; i++) {
            if (hurdles[i][0] == x && hurdles[i][1] == y) {
                return true;
            }
        }
        return false;
    }
    public String hurdleList() {
        String text = "";
        for (int i = 0; i < hurdles.length; i++) {
            text = text + "[" + hurdles[i][0] + ":" + hurdles[i][1] + "] ";
        }
        return text;
    }
}
class DirectedWalker {
    private String name;
    private int x = 0;
    private int y = 0;
    private Field field;
    public DirectedWalker(String name, Field field) {
        this.name = name;
        this.field = field;
    }
    public String where() {
        return x + " : " + y;
    }
    public boolean walk(String order) {
        if (order.equals("^")) {
            move(0, 1);
        } else if (order.equals("v")) {
            move(0, -1);
        } else if (order.equals("<")) {
            move(-1, 0);
        } else if (order.equals(">")) {
            move(1, 0);
        } else {
            return false;
        }
        return true;
    }
    private void move(int dx, int dy) {
        int newX = x + dx;
        int newY = y + dy;
        if (field.isHurdle(newX, newY)) {
            System.out.println(name + " encounters hurdle at [" + newX + ":" + newY + "]");
        } else {
            x = newX;
            y = newY;
        }
```

```
        System.out.println(name + " is at = " + where());
    }
}
class RandomWalker {
    private String name;
    private int x = 0;
    private int y = 0;
    private Field field;
    private Random random = new Random();
    public RandomWalker(String name, Field field) {
        this.name = name;
        this.field = field;
    }
    public String where() {
        return x + " : " + y;
    }
    public void walkRandomly() {
        int direction = random.nextInt(4);
        if (direction == 0) {
            move(0, 1);
        } else if (direction == 1) {
            move(0, -1);
        } else if (direction == 2) {
            move(-1, 0);
        } else {
            move(1, 0);
        }
    }
    private void move(int dx, int dy) {
        int newX = x + dx;
        int newY = y + dy;
        if (field.isHurdle(newX, newY)) {
            System.out.println(name + " encounters hurdle at [" + newX + ":" + newY + "]");
        } else {
            x = newX;
            y = newY;
        }
        System.out.println(name + " is at = " + where());
    }
}
public class Task_02 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Field field = new Field();
        DirectedWalker ali = new DirectedWalker("Ali", field);
        RandomWalker hasan = new RandomWalker("Hasan", field);
        System.out.println("Hurdles are at: " + field.hurdleList());
        System.out.println("Starting: Ali's Location is = " + ali.where() + "  Hasan's Location is = "
+ hasan.where());
        System.out.println("Hasan has Random Movement, Ali follows order (^ v < >, q to quit)");
        while (true) {
            System.out.print("\nAli: Where should I go now: ");
            if (!sc.hasNextLine()) {
                break;
            }
            String order = sc.nextLine().trim();
            if (order.equals("q")) {
                System.out.println("Simulation finished.");
                break;
            }
            boolean ok = ali.walk(order);
            if (!ok) {
```

```
            System.out.println("Please type ^ v < > or q");
            continue;
        }
        hasan.walkRandomly();
    }
  }
}
```

Output:

```
Hurdles are at: [1:2] [-1:1] [2:-1] [0:-3] [-2:-2]
Starting: Ali's Location is = 0 : 0  Hasan's Location is = 0 : 0
Hasan has Random Movement, Ali follows order (^ v < >, q to quit)

Ali: Where should I go now:
```

```
Ali: Where should I go now: v
Ali is at = 0 : -1
Hasan is at = 0 : -1

Ali: Where should I go now: >
Ali is at = 1 : -1
Hasan is at = -1 : -1

Ali: Where should I go now: >
Ali encounters hurdle at [2:-1]
Ali is at = 1 : -1
Hasan is at = 0 : -1

Ali: Where should I go now: >
Ali encounters hurdle at [2:-1]
Ali is at = 1 : -1
Hasan is at = 0 : -2

Ali: Where should I go now: >
Ali encounters hurdle at [2:-1]
Ali is at = 1 : -1
Hasan is at = 1 : -2
```

```
Ali: Where should I go now: q
Simulation finished.
```